AMBA AHB-to-APB Bridge

Design, Verification, and Synthesis

Internship Project Report

Submitted by:
Vallabh Gondkar
Batch: DI 66

Submitted To:
Maven Silicon

Contents

1 Architecture **2**

2 Protocol Information **2**

3 Top Module Block Diagram - Overall Functionality **2**

4 Sub-blocks Block Diagram - Functionality **3**

5 Output Waveforms (Top Module & Sub-blocks) **3**

6 Synthesis Results **4**

7 Conclusion **5**

A Appendix: Core RTL Codes **6**
A.1 Bridge_Top.v 6
A.2 AHB_slave_interface.v 7
A.3 APB_FSM_Controller.v 7

1 Architecture

The AMBA AHB-to-APB bridge operates as a slave on the Advanced High-performance Bus (AHB) and as a master on the Advanced Peripheral Bus (APB). The core architecture is specifically designed to manage the fundamental pipeline differences between the two bus protocols. It absorbs the high-speed, pipelined address and data phases of AHB burst transfers and safely converts them into the sequential, non-pipelined two-cycle (SETUP and ENABLE) transfers required by the APB peripherals, ensuring zero data loss and protocol compliance.

2 Protocol Information

The project implements a bridge between two distinct AMBA (Advanced Microcontroller Bus Architecture) protocols:

- **AHB (Advanced High-performance Bus):** A high-performance, pipelined bus designed for high clock frequency system modules. It operates using a two-phase transfer: an address phase and a subsequent data phase. It supports burst transfers (INCR, WRAP) and uses a ready signal (`HREADY`) to extend transfers.
- **APB (Advanced Peripheral Bus):** A simpler, unpipelined bus designed for low-bandwidth peripherals to minimize power consumption and interface complexity. It operates strictly in two cycles: a SETUP phase (where address and select signals are asserted) and an ENABLE phase (where the data transfer occurs and the enable signal is asserted).

The core function of this bridge is to absorb the pipelined, high-speed AHB burst transfers and convert them into the sequential, two-cycle APB transfers without data loss.

3 Top Module Block Diagram - Overall Functionality

The top module (`Bridge_Top`) serves as the structural wrapper that connects the AHB and APB domains. Its overall functionality is to monitor the AHB interface for incoming read/write requests, route the pipelined addresses and data through a slave interface, and trigger a state machine to drive the APB peripheral interface.

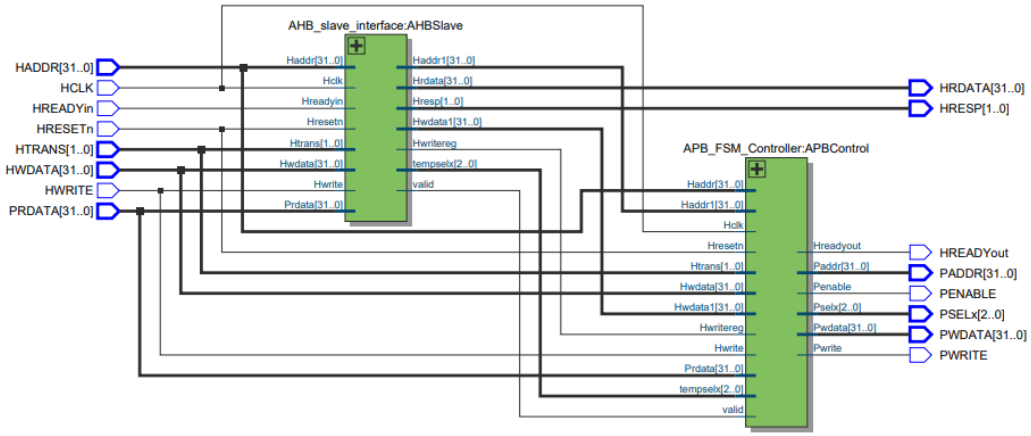

Figure 1: RTL Schematic showing the Top Module housing the interconnected sub-blocks.

4 Sub-blocks Block Diagram - Functionality

As illustrated in the schematic above, the architecture is divided into two primary sub-blocks:

- **AHB Slave Interface (`AHBSlave`):** Monitors the AHB bus and decodes incoming addresses to generate the slave select (`tempselx`). Its primary functionality is to implement pipeline-delayed registers (`Haddr1`, `Hwdata1`) to align the AHB master's pipelined address phase with its subsequent data phase. It utilizes the bridge's `Hreadyin` feedback to ensure valid transfers are only accepted when the bus is free, preventing combinational loops.
- **APB FSM Controller (`APBControl`):** An 8-state Moore Finite State Machine. Its functionality is to handle the protocol conversion. It captures the aligned AHB address and data, then systematically transitions through the `ST_SETUP` and `ST_ENABLE` states to drive the APB `PSELx`, `PENABLE`, and `PWRITE` signals, fulfilling the APB protocol timing requirements.

5 Output Waveforms (Top Module & Sub-blocks)

The design underwent rigorous verification using ModelSim to ensure absolute protocol compliance. The waveform below captures both the Top Module boundary signals (AHB inputs and APB outputs) as well as the internal Sub-block bridging signals (`valid`, `Haddr1`, `Hwdata1`, and the FSM `PRESENT_STATE`).

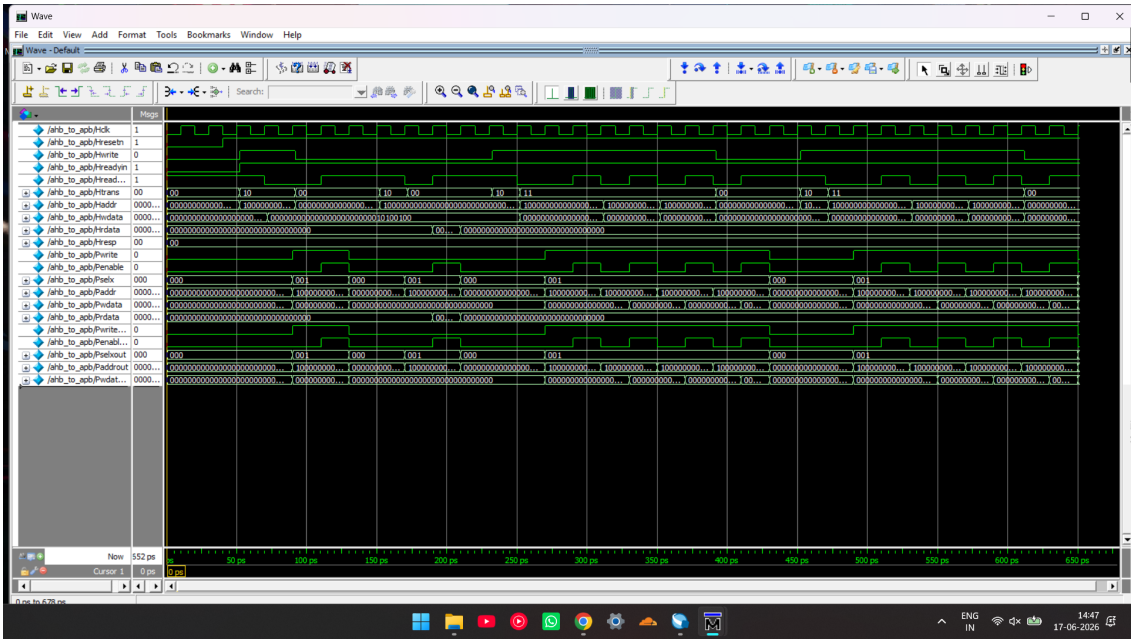

Figure 2: ModelSim waveform demonstrating Top Module I/O and internal Sub-block pipeline alignment for Single, INCR4, and WRAP4 transfers.

The waveform confirms that the FSM correctly stalls the master, absorbs the pipeline lag, and guarantees that consecutive burst beats perfectly align their corresponding address and data bytes across both the AHB Slave sub-block and the APB FSM sub-block without skipping beats.

6 Synthesis Results

The RTL was synthesized targeting the Intel MAX 10 FPGA using Quartus Prime to confirm physical hardware viability.

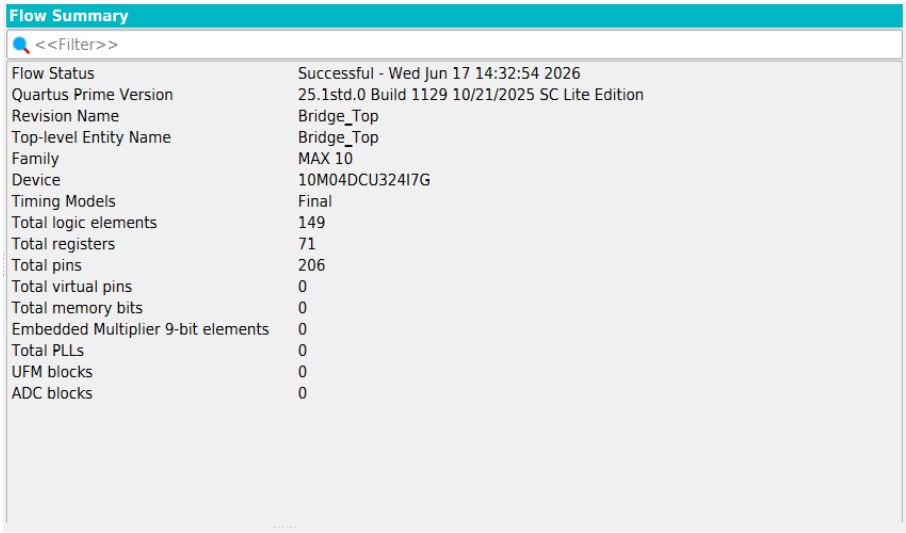

Figure 3: Quartus Prime Flow Summary

The synthesis compiler confirmed a robust, latch-free elaboration utilizing exactly 149 Total Logic Elements and 71 Total Registers.

7 Conclusion

This project successfully implemented a fully synthesizable, AMBA-compliant AHB-to-APB bridge. Critical timing hazards—such as zero-delay combinational feedback loops and pipeline data-shifting during burst transfers—were systematically resolved. By implementing a strict 8-state FSM, the sub-blocks correctly capture AHB address phases when the bus is free and data phases during master stall cycles, completely eliminating data loss and ensuring perfect alignment for Single Writes and Bursts.

A Appendix: Core RTL Codes

A.1 Bridge_Top.v

```
module Bridge_Top(
    input  HCLK, HRESETn,
    input  HWRITE, HREADYin,
    input  [31:0] HADDR, HWDATA,
    input  [1:0]  HTRANS,
    output        HREADYout,
    output [31:0] HRDATA,
    output [1:0]  HRESP,
    input  [31:0] PRDATA,
    output        PWRITE, PENABLE,
    output [2:0]  PSELx,
    output [31:0] PADDR, PWDATA
);

    wire [31:0] Haddr1, Hwdata1;
    wire        valid, Hwritereg;
    wire [2:0]  tempselx;

    AHB_slave_interface AHBSlave (
        .Hclk      (HCLK),
        .Hresetn   (HRESETn),
        .Hwrite    (HWRITE),
        .Hreadyin  (HREADYin),
        .Haddr     (HADDR),
        .Hwdata    (HWDATA),
        .Prdata    (PRDATA),
        .Htrans    (HTRANS),
        .valid     (valid),
        .Hwritereg (Hwritereg),
        .Haddr1    (Haddr1),
        .Hwdata1   (Hwdata1),
        .Hrdata    (HRDATA),
        .tempselx  (tempselx),
        .Hresp     (HRESP)
    );

    APB_FSM_Controller APBControl (
        .Hclk      (HCLK),
        .Hresetn   (HRESETn),
        .valid     (valid),
        .Hwrite    (HWRITE),
        .Hwritereg (Hwritereg),
        .Haddr     (HADDR),
        .Haddr1    (Haddr1),
        .Hwdata    (HWDATA),
        .Hwdata1   (Hwdata1),
        .Prdata    (PRDATA),
        .Htrans    (HTRANS),
        .tempselx  (tempselx),
        .Pwrite    (PWRITE),
```

```
        .Penable    (PENABLE),
        .Hreadyout  (HREADYout),
        .Pselx      (PSELx),
        .Paddr      (PADDR),
        .Pwdata     (PWDATA)
    );
endmodule
```

A.2 AHB_slave_interface.v

```
module AHB_slave_interface(
    input Hclk, Hresetn, Hwrite, Hreadyin,
    input [31:0] Haddr, Hwdata, Prdata,
    input [1:0] Htrans,
    output reg valid, Hwritereg,
    output reg [31:0] Haddr1, Hwdata1,
    output [31:0] Hrdata,
    output reg [2:0] tempselx,
    output [1:0] Hresp
);
    assign Hrdata = Prdata;
    assign Hresp = 2'b00;

    always @(posedge Hclk or negedge Hresetn) begin
        if (~Hresetn) begin
            Haddr1 <= 0; Hwdata1 <= 0; Hwritereg <= 0;
        end else if (Hreadyin) begin
            Haddr1 <= Haddr;
            Hwdata1 <= Hwdata;
            Hwritereg <= Hwrite;
        end
    end

    always @(*) begin
        valid = 0; tempselx = 0;
        if (Haddr >= 32'h8000_0000 && Haddr < 32'h8000_0400) begin
            tempselx = 3'b001;
            if (Hreadyin && (Htrans == 2'b10 || Htrans == 2'b11))
   begin
                valid = 1'b1;
            end
        end
    end
endmodule
```

A.3 APB_FSM_Controller.v

```
module APB_FSM_Controller(
    input Hclk, Hresetn, valid, Hwrite, Hwritereg,
    input [31:0] Haddr, Haddr1, Hwdata, Hwdata1, Prdata,
    input [1:0] Htrans,
    input [2:0] tempselx,
```

```
    output reg Pwrite, Penable, Hreadyout,
    output reg [2:0] Pselx,
    output reg [31:0] Paddr, Pwdata
);
    parameter ST_IDLE = 3'b000, ST_WWAIT = 3'b001, ST_READ = 3'b010,
              ST_WRITE = 3'b011, ST_WRITEP = 3'b100, ST_RENABLE = 3'b101,
              ST_WENABLE = 3'b110, ST_WENABLEP = 3'b111;

    reg [2:0] PRESENT_STATE, NEXT_STATE;

    always @(posedge Hclk or negedge Hresetn) begin
        if (~Hresetn) PRESENT_STATE <= ST_IDLE;
        else PRESENT_STATE <= NEXT_STATE;
    end

    always @(*) begin
        NEXT_STATE = ST_IDLE;
        case(PRESENT_STATE)
            ST_IDLE: begin
                if (valid && Hwrite) NEXT_STATE = ST_WWAIT;
                else if (valid && ~Hwrite) NEXT_STATE = ST_READ;
                else NEXT_STATE = ST_IDLE;
            end
            ST_WWAIT:  NEXT_STATE = ST_WRITE;
            ST_READ:   NEXT_STATE = ST_RENABLE;
            ST_WRITE:  NEXT_STATE = ST_WENABLE;
            ST_WRITEP: NEXT_STATE = ST_WENABLEP;
            ST_RENABLE, ST_WENABLE, ST_WENABLEP: begin
                if (valid && ~Hwrite) NEXT_STATE = ST_READ;
                else if (valid && Hwrite) NEXT_STATE = ST_WRITEP;
                else NEXT_STATE = ST_IDLE;
            end
            default: NEXT_STATE = ST_IDLE;
        endcase
    end

    always @(*) begin
        case(PRESENT_STATE)
            ST_IDLE, ST_RENABLE, ST_WENABLE, ST_WENABLEP: Hreadyout = 1'b1;
            default: Hreadyout = 1'b0;
        endcase
    end

    reg [31:0] paddr_reg, pwdata_reg;
    reg [2:0] pselx_reg;

    always @(posedge Hclk or negedge Hresetn) begin
        if (~Hresetn) begin
            paddr_reg <= 0; pwdata_reg <= 0; pselx_reg <= 0;
        end else begin
            if ((PRESENT_STATE == ST_IDLE || PRESENT_STATE == ST_RENABLE ||
```

```
                    PRESENT_STATE == ST_WENABLE || PRESENT_STATE ==
    ST_WENABLEP) && valid) begin
                paddr_reg <= Haddr;
                pselx_reg <= tempselx;
            end

            if (PRESENT_STATE == ST_WWAIT || PRESENT_STATE ==
    ST_WRITEP) begin
                pwdata_reg <= Hwdata;
            end
        end
     end

     always @(*) begin
        Pwrite = 1'b0; Penable = 1'b0; Pselx = 3'b000; Paddr = 32'
    h0; Pwdata = 32'h0;
        case(PRESENT_STATE)
            ST_READ: begin Pselx = pselx_reg; Paddr = paddr_reg;
    Pwrite = 1'b0; Penable = 1'b0; end
            ST_RENABLE: begin Pselx = pselx_reg; Paddr = paddr_reg;
     Pwrite = 1'b0; Penable = 1'b1; end
            ST_WRITE, ST_WRITEP: begin Pselx = pselx_reg; Paddr =
    paddr_reg; Pwdata = pwdata_reg; Pwrite = 1'b1; Penable = 1'b0;
    end
            ST_WENABLE, ST_WENABLEP: begin Pselx = pselx_reg; Paddr
     = paddr_reg; Pwdata = pwdata_reg; Pwrite = 1'b1; Penable = 1'b1
    ; end
        endcase
     end
endmodule
```